CYBERSECURITY
ASSESSMENT REPORT

Vulnerability Assessment & Penetration Testing Report

Web Application Security Assessment

Target Web Application	https://testfire.net/
Assessment Date	September 20, 2026
Prepared By	Nikhil Gowda
Testing Methodology	Manual & Automated VAPT
Submitted To	Rahul Chauhan (Mrwebsecure Infosolutions)

CONFIDENTIAL

This document contains assessment observations and technical evidence intended for the stated recipient. Findings are based on the testing described in this report.

1. EXECUTIVE SUMMARY

A comprehensive security assessment was conducted on the Altoro Mutual web application to identify security weaknesses across authentication, input validation, access control, and configuration management layers. A total of 10 distinct vulnerabilities ranging from Low to High severity were successfully discovered and verified.

Assessment at a Glance

10 Findings	**1** Critical in summary	**4** High in summary	**4** Medium in summary	**1** Low in summary

2. SUMMARY OF FINDINGS

S.No.	Vulnerability Name	Category	Severity
1	SQL Injection	Injection	**Critical**
2	Broken Access Control / BOLA / IDOR	Broken Access Control	**High**
3	Cross-Site Request Forgery (CSRF)	Broken Access Control	**High**
4	Reflected Cross-Site Scripting (XSS)	Injection	**High**
5	HTML Injection	Injection	**Medium**
6	Information Disclosure / Application Error Handling Flaw	Security Misconfiguration	**Medium**
7	Hardcoded Administrative Password	Identification and Authentication Failures	**High**
8	Sensitive Cookie Without HttpOnly Flag	Cryptographic Failures / Security Misconfiguration	**Medium**
9	Missing Security Headers / Misconfiguration	Security Misconfiguration	**Low**
10	Missing Rate Limiting / Brute-Force Vulnerability	Security Misconfiguration	**Medium**

Severity note: The report preserves the severity values as stated in the original summary and detailed finding sections. Where those sections differ, both are retained rather than silently reconciled.

3. ASSESSMENT SCOPE & METHODOLOGY

Target: https://testfire.net/ (Altoro Mutual web application)
Assessment date: September 20, 2026
Approach: Manual & Automated VAPT

The assessment focused on application security weaknesses involving authentication, authorization, input handling, session management, error handling, response headers, and brute-force protections. The findings below retain the observed endpoints, impacts, proof-of-concept steps, remediation guidance, and CWE identifiers documented in the source report.

Tools and techniques referenced

Burp Suite, browser Developer Tools, HTTP request/response inspection, browser page-source inspection, and Kali Linux command-line checks (including curl) are referenced in the individual proof-of-concept procedures.

Report structure

Each finding includes the vulnerability name/type, affected parameter or endpoint where supplied, risk rating, impact, remediation, CWE identifier, step-by-step PoC, and available screenshot evidence extracted from the source PDF.

4.1 SQL Injection (SQLi)

Vulnerability Name	SQL Injection resulting in Authentication Bypass
Vulnerability Type	Injection / Broken Authentication
Affected Parameter / Endpoint	Username field on /login.jsp
Affected URL / Endpoint	https://testfire.net/login.jsp
Risk Rating	**Critical**
Impact	A severe vulnerability that allows an unauthenticated attacker to bypass login controls. An attacker can gain unauthorized access to any user account (including administrator accounts), access sensitive data (bank details), and potentially perform transactions or change account settings.
Remediation	Implement parameterized queries (Prepared Statements) and Stored Procedures.
CWE ID	CWE-89 (Improper Neutralization of Special Elements used in an SQL Command)

Step-by-Step Proof of Concept (PoC)

1. Navigate to the target URL: https://testfire.net/login.jsp.

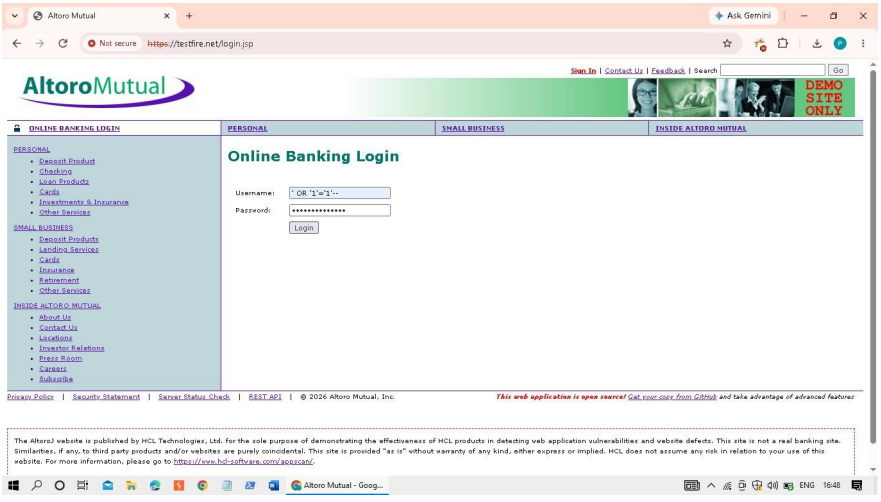
2. In the 'Username' field, enter the following SQL injection payload: `' OR '1'='1'-- ` .

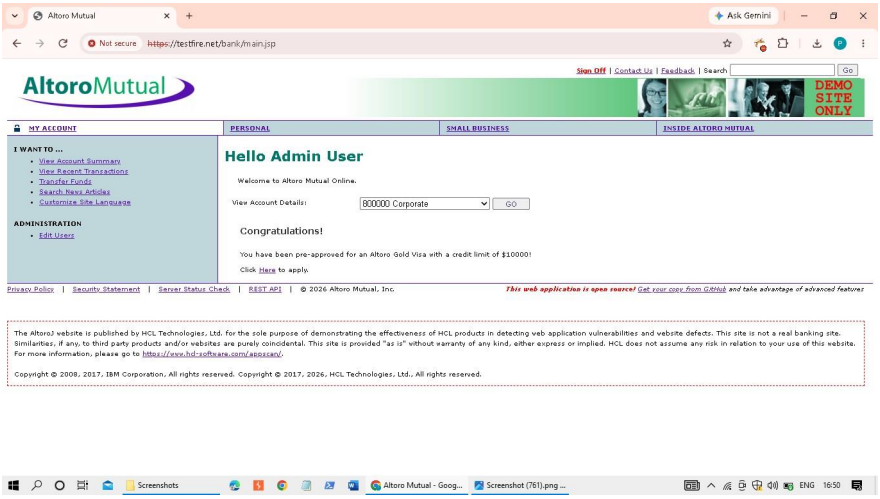
3. Enter any string in the 'Password' field (e.g., anything).

4. Click the 'Login' button.

5. Observation: The application authenticates you and directs you to the account dashboard with "Hello Admin User", demonstrating that the login security is bypassed.

Screenshot Evidence




Altoro Mutual
Not secure https://testfire.net/bank/main.jsp
Sign Off | Contact Us | Feedback | Search
Go
AltoroMutual
DEMO SITE ONLY
MY ACCOUNT
PERSONAL
SMALL BUSINESS
INSIDE ALTORO MUTUAL
I WANT TO ...
View Account Summary
View Recent Transactions
Transfer Funds
Search News Articles
Customize Site Language
ADMINISTRATION
Edit Users
Hello Admin User
Welcome to Altoro Mutual Online.
View Account Details:
800000 Corporate
GO
Congratulations!
You have been pre-approved for an Altoro Gold Visa with a credit limit of $10000!
Click Here to apply.
Privacy Policy | Security Statement | Server Status Check | REST API | © 2026 Altoro Mutual, Inc.
This web application is open source! Get your copy from GitHub and take advantage of advanced features
The Altoro Mutual website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to https://www.hcl-software.com/appscan/.
Copyright © 2008, 2017, IBM Corporation, All rights reserved. Copyright © 2017, 2026, HCL Technologies, Ltd., All rights reserved.
Screenshots
Altoro Mutual - Goog...
Screenshot (761).png ...
ENG
16:50

4.2 Broken Access Control / BOLA / IDOR

Vulnerability Name	Insecure Direct Object Reference (IDOR) / Broken Object Level Authorization (BOLA)
Vulnerability Type	Broken Access Control
Affected Parameter / Endpoint	listAccounts (GET Parameter)
Affected URL / Endpoint	https://testfire.net/bank/showAccount?listAccounts=800001
Risk Rating	**High**
Impact	Any authenticated user can view balance details, recent transactions, and account histories of other customers simply by changing the account ID in the URL parameter.
Remediation	Enforce strict server-side authorization checks verifying that the currently authenticated session owns the requested resource.
CWE ID	CWE-639 (Authorization Bypass Through User-Controlled Key) / CWE-284 (Improper Access Control)

Step-by-Step Proof of Concept (PoC)

1. Login to the application and navigate to the View Account Details page (https://testfire.net/bank/main.jsp).

2. Select an account (e.g., 800000) and submit.

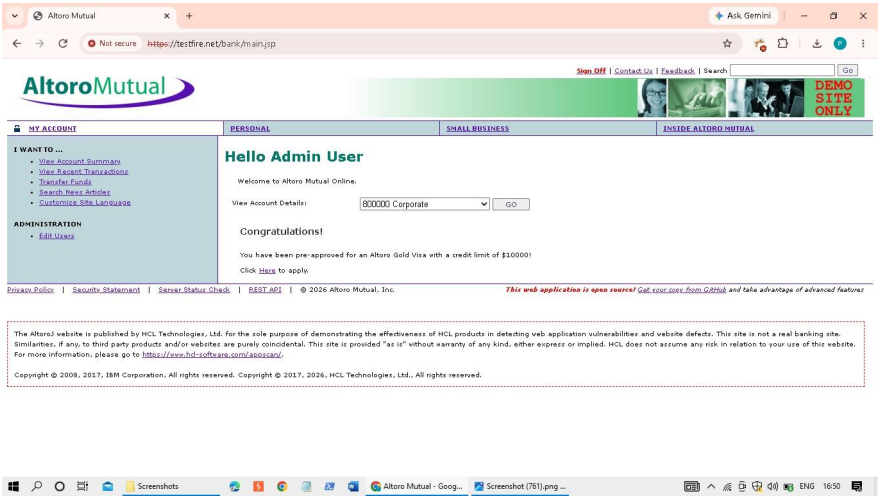
3. Observe the URL in the browser address bar: https://testfire.net/bank/showAccount?listAccounts=800000.

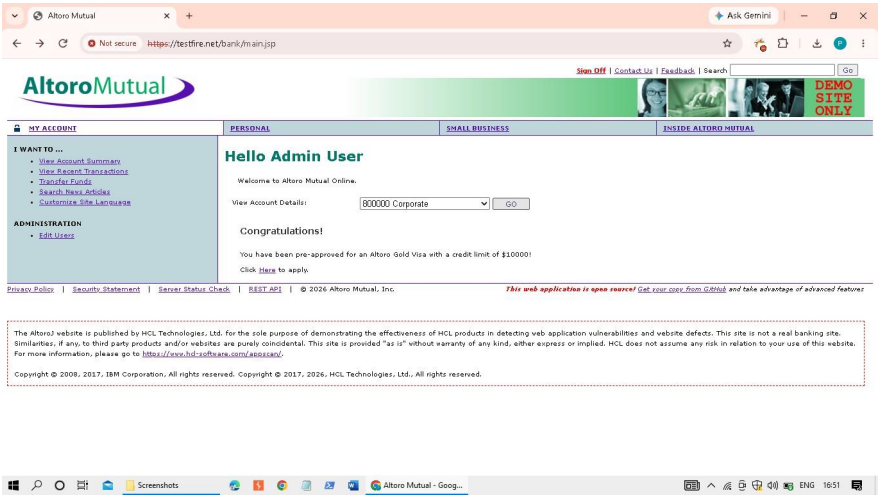
4. Modify the listAccounts query parameter value to another account ID, such as 800001.

5. Press Enter to load the page (https://testfire.net/bank/showAccount?listAccounts=800001).

6. Observation: The application displays the bank balance and transaction history of account 800001 Checking without checking if the logged-in user owns that account.

Screenshot Evidence




Altoro Mutual
Not secure https://testfire.net/bank/main.jsp
Sign Off | Contact Us | Feedback | Search
Go
AltoroMutual
DEMO SITE ONLY
MY ACCOUNT
PERSONAL
SMALL BUSINESS
INSIDE ALTORO MUTUAL
I WANT TO ...
View Account Summary
View Recent Transactions
Transfer Funds
Search News Articles
Customize Site Language
ADMINISTRATION
Edit Users
Hello Admin User
Welcome to Altoro Mutual Online.
View Account Details:
800000 Corporate
GO
Congratulations!
You have been pre-approved for an Altoro Gold Visa with a credit limit of $10000!
Click Here to apply.
Privacy Policy | Security Statement | Server Status Check | REST API | © 2026 Altoro Mutual, Inc.
This web application is open source! Get your copy from GitHub and take advantage of advanced features
The AltoroJ website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to https://www.hcl-software.com/appscan/.
Copyright © 2008, 2017, IBM Corporation, All rights reserved. Copyright © 2017, 2026, HCL Technologies, Ltd., All rights reserved.
Screenshots
Altoro Mutual - Goog...
ENG 16:51

4.3 Cross-Site Request Forgery (CSRF)

Vulnerability Name	Missing Anti-CSRF Token on Financial State-Changing Request
Vulnerability Type	Cross-Site Request Forgery (CSRF)
Affected Parameter / Endpoint	POST /bank/doTransfer
Affected URL / Endpoint	http://testfire.net/bank/doTransfer
Risk Rating	**High**
Impact	The source report identifies a missing unique, unpredictable CSRF token on a financial state-changing request. It also notes that the application allows transferring money from Account 800000 to the same Account 800000 without any server validation error.
Remediation	Implement unique Anti-CSRF tokens per session/request and enforce the SameSite=Strict or Lax cookie attribute.
CWE ID	CWE-352 (Cross-Site Request Forgery) & CWE-840 (Business Logic Errors)

Step-by-Step Proof of Concept (PoC)

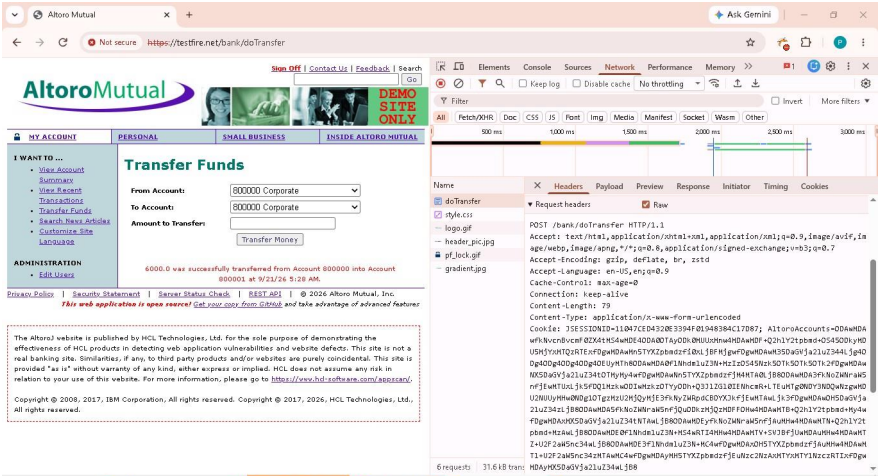
1. Navigate to the transfer page: http://testfire.net/bank/doTransfer.

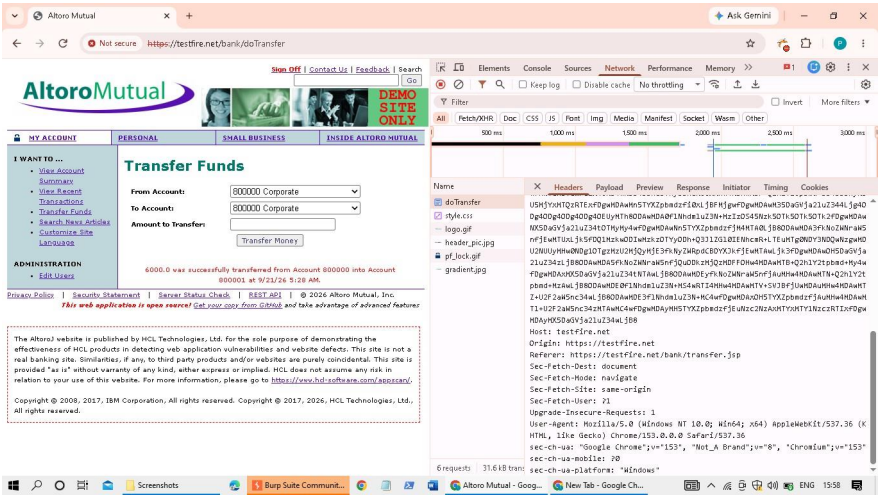
2. Submit a fund transfer request and capture/inspect the POST request using DevTools or Burp Suite.

3. Observation: Note that Content-Type is application/x-www-form-urlencoded and no unique, unpredictable CSRF token is present in the parameters or headers.

4. Additionally, observe that the application allows transferring money from Account 800000 to the same Account 800000 without any server validation error.

Screenshot Evidence




Altoro Mutual
Not secure https://testfire.net/bank/doTransfer
Sign Off | Contact Us | Feedback | Search
AltoroMutual
DEMO SITE ONLY
MY ACCOUNT | PERSONAL | SMALL BUSINESS | INSIDE ALTORO MUTUAL
I WANT TO ...
View Account Summary
View Recent Transactions
Transfer Funds
Search News Articles
Customize Site Language
ADMINISTRATION
Edit Users
Transfer Funds
From Account: 800000 Corporate
To Account: 800000 Corporate
Amount to Transfer:
Transfer Money
6000.0 was successfully transferred from Account 800000 into Account 800001 at 9/21/24 5:28 AM.
Privacy Policy | Security Statement | Server Status Check | REST API | © 2026 Altoro Mutual, Inc.
This web application is open source! Get your copy from GitHub and take advantage of advanced features
The Altoro Mutual website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to https://www.hcl-software.com/appscan/.
Copyright © 2008, 2017, IBM Corporation, All rights reserved. Copyright © 2017, 2026, HCL Technologies, Ltd., All rights reserved.
Elements Console Sources Network Performance Memory
Keep log Disable cache No throttling
Filter
Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other
Name
doTransfer
style.css
logo.gif
header_pic.jpg
pf_lock.gif
gradient.jpg
Headers Payload Preview Response Initiator Timing Cookies
Origin: https://testfire.net
Referer: https://testfire.net/bank/transfer.jsp
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36
sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
6 requests
Screenshots
Burp Suite Communit...
Altoro Mutual - Goog...
New Tab - Google Ch...
ENG 15:58

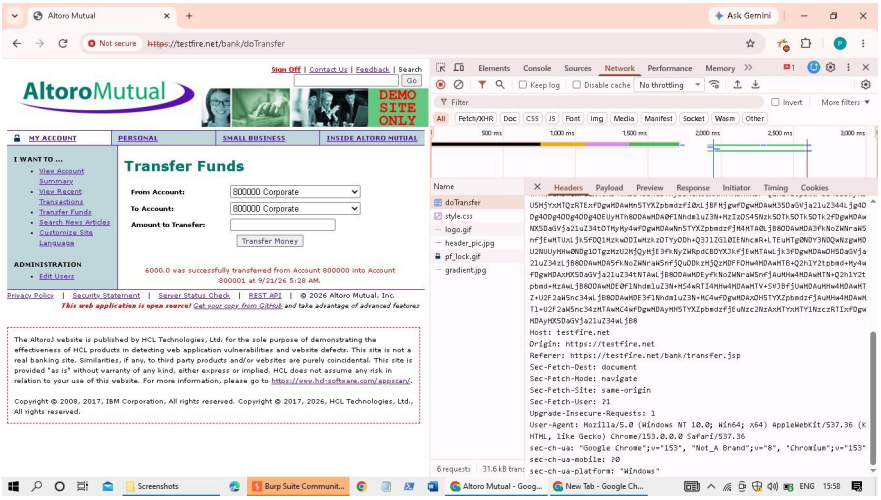
4.4 Reflected Cross-Site Scripting (Reflected XSS)

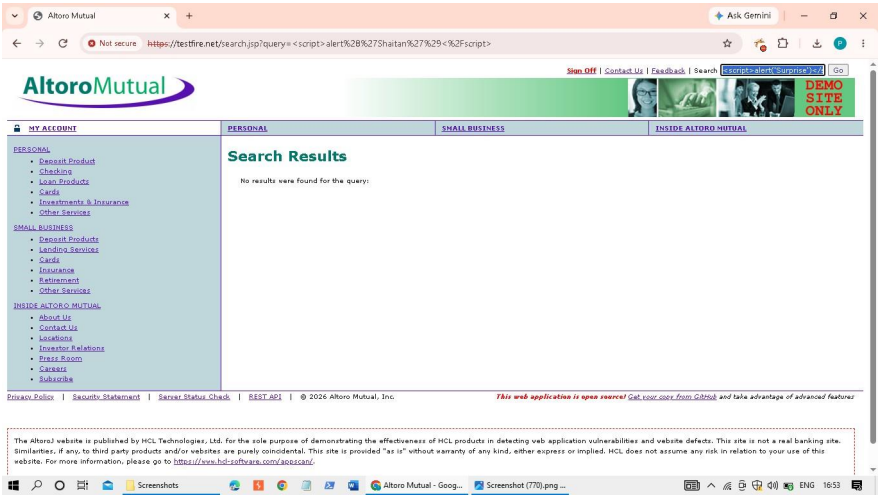
Vulnerability Name	Reflected Cross-Site Scripting (Reflected XSS)
Vulnerability Type	Client-Side Injection / Cross-Site Scripting
Affected Parameter / Endpoint	query (GET Parameter)
Affected URL / Endpoint	https://testfire.net/search.jsp?query=
Risk Rating	**Medium**
Severity note	Summary table: High; detailed finding: Medium.
Impact	An attacker can craft a malicious link containing the XSS payload and send it to victim users. If an authenticated user clicks the link, the attacker can hijack session cookies, redirect the user to phishing sites, or perform actions on their behalf.
Remediation	Perform contextual output encoding and input validation for all user-supplied data rendering in the browser.
CWE ID	CWE-79 (Improper Neutralization of Input During Web Page Generation / Cross-Site Scripting)

Step-by-Step Proof of Concept (PoC)

1. Open the web application URL: https://testfire.net/search.jsp.
2. Locate the search bar on the top right corner of the page.
3. Input the following XSS payload into the search field: `<script>alert('Surprise')</script>`.
4. Click the 'Go' button or press Enter.
5. Observation: The browser executes the injected JavaScript payload and displays an alert box containing "Surprise".

Screenshot Evidence




Altoro Mutual
Not secure
https://testfire.net/search.jsp?query=<script>alert%28%27Shaitan%27%29<%2Fscript>
Sign Off | Contact Us | Feedback | Search
<script>alert('Surprise')</
Go
AltoroMutual
DEMO SITE ONLY
MY ACCOUNT
PERSONAL
SMALL BUSINESS
INSIDE ALTORO MUTUAL
PERSONAL
Deposit Product
Checking
Loan Products
Cards
Investments & Insurance
Other Services
SMALL BUSINESS
Deposit Products
Lending Services
Cards
Insurance
Retirement
Other Services
INSIDE ALTORO MUTUAL
About Us
Contact Us
Locations
Investor Relations
Press Room
Careers
Subscribe
Search Results
No results were found for the query:
Privacy Policy | Security Statement | Server Status Check | REST API | © 2026 Altoro Mutual, Inc.
This web application is open source! Get your copy from GitHub and take advantage of advanced features
The AltoroJ website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to https://www.hcl-software.com/appscan/.
Screenshots
Altoro Mutual - Goog...
Screenshot (770).png ...
ENG
16:53

4.5 HTML Injection

Vulnerability Name	HTML Injection
Vulnerability Type	Client-Side Code Injection
Affected Parameter / Endpoint	query parameter on /search.jsp
Affected URL / Endpoint	https://testfire.net/search.jsp
Risk Rating	**Medium**
Impact	Attackers can deface page content, alter UI layout, or craft malicious URLs to trick users into phishing or executing arbitrary scripts in their browsers.
Remediation	Escape special HTML characters (<, >, &, ", ') before output rendering.
CWE ID	Not specified in the source report.

Step-by-Step Proof of Concept (PoC)

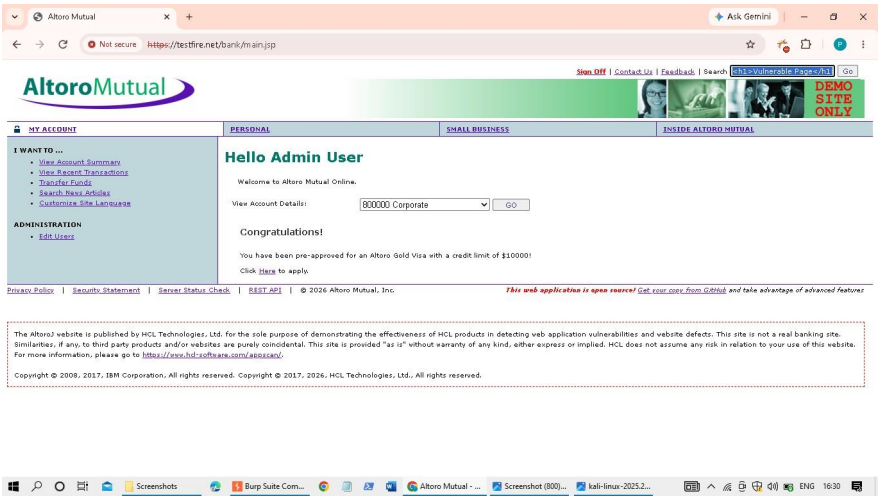
1. Open the web application and locate the search bar: https://testfire.net/bank/main.jsp.

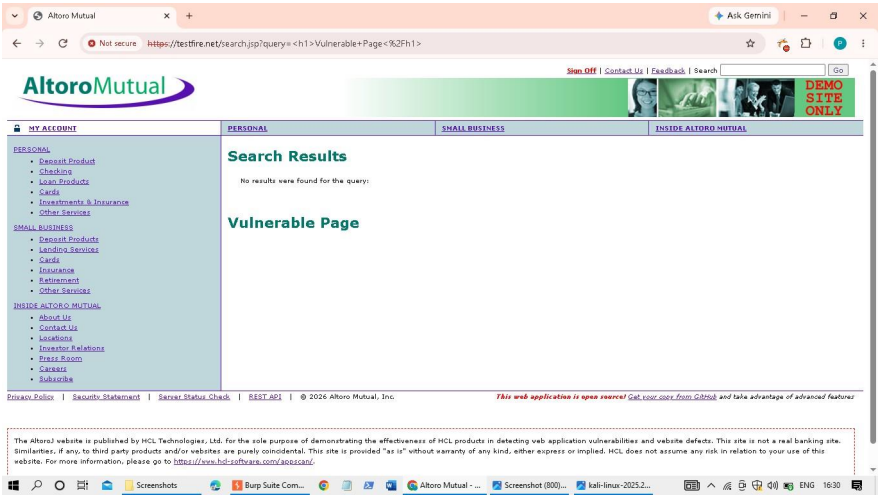
2. Enter the HTML payload into the search input: `<h1>Vulnerable Page</h1>`.

3. Click on the 'Go' button.

4. Observation: The browser redirects to /search.jsp?query=... where the injected payload gets parsed as valid HTML and rendered as styled heading text on the webpage.

Screenshot Evidence




Altoro Mutual
Not secure https://testfire.net/search.jsp?query=<h1>Vulnerable+Page<%2Fh1>
Sign Off | Contact Us | Feedback | Search
Go
AltoroMutual
DEMO SITE ONLY
MY ACCOUNT
PERSONAL
SMALL BUSINESS
INSIDE ALTORO MUTUAL
Search Results
No results were found for the query:
Vulnerable Page
Privacy Policy | Security Statement | Server Status Check | REST API | © 2026 Altoro Mutual, Inc.
This web application is open source! Get your copy from GitHub and take advantage of advanced features
The AltoroJ website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real banking site. Similarities, if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to https://www.hcl-software.com/appscan/.
Screenshots
Burp Suite Com...
Altoro Mutual - ...
Screenshot (800)...
kali-linux-2025.2...
ENG
16:30

4.6 Information Disclosure / Application Error Handling Flaw

Vulnerability Name	Verbose Error Message / Information Disclosure via Stack Trace
Vulnerability Type	Not separately specified
Affected Parameter / Endpoint	listAccounts parameter on /bank/showAccount
Affected URL / Endpoint	https://testfire.net/bank/showAccount?listAccounts=800000
Risk Rating	**Medium**
Impact	Attackers can analyze the internal technology stack, software versions, source code structure, and package naming conventions to plan subsequent targeted attacks (such as SQL Injection or Remote Code Execution).
Remediation	Disable detailed stack traces in production environments and implement centralized custom error pages (e.g., generic 404/500 error pages).
CWE ID	CWE-209 (Generation of Error Message Containing Sensitive Information)

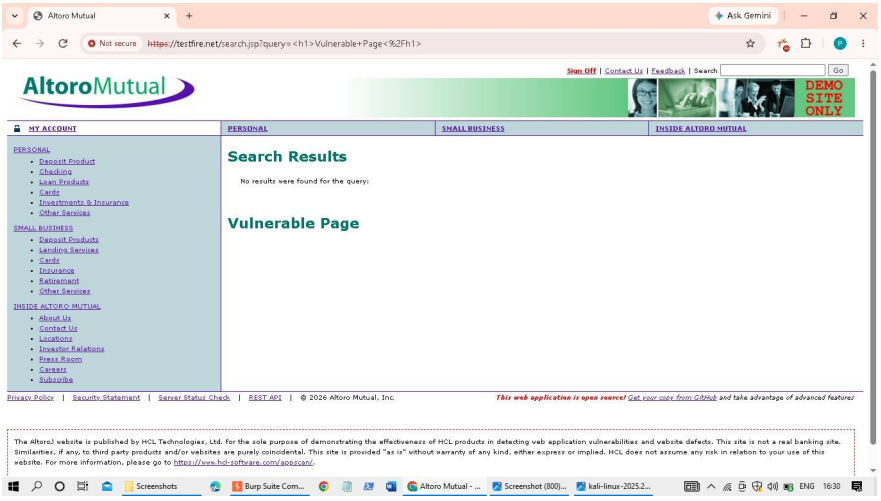
Step-by-Step Proof of Concept (PoC)

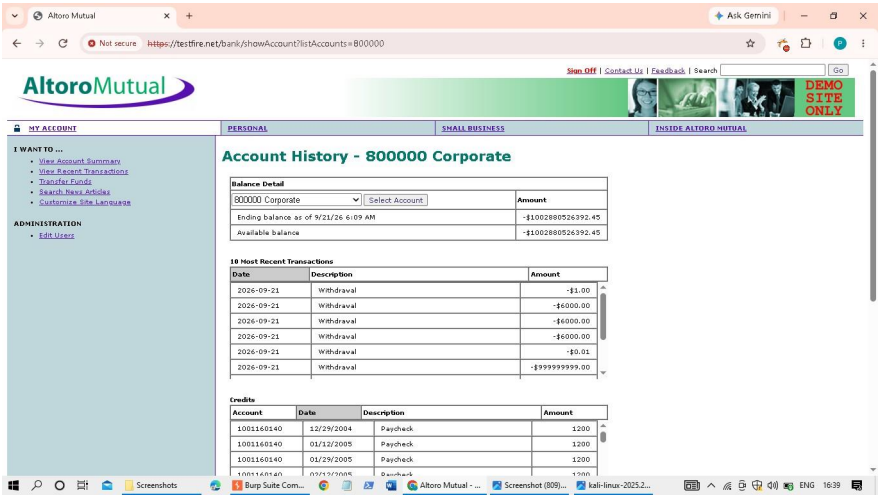
1. Navigate to the account viewer endpoint: https://testfire.net/bank/showAccount?listAccounts=800000.

2. Modify the parameter value in the URL to inject an unexpected character, such as listAccounts=%27 (single quote) or leave it empty.

3. Observation: The server crashes with an HTTP Status 500 error and dumps detailed Java exception logs, including java.lang.NumberFormatException and internal JSP file paths directly onto the user's browser.

Screenshot Evidence




Altoro Mutual
Not secure https://testfire.net/bank/showAccount?listAccounts=800000
Sign Off | Contact Us | Feedback | Search
Go
AltoroMutual
DEMO SITE ONLY
MY ACCOUNT
PERSONAL
SMALL BUSINESS
INSIDE ALTORO MUTUAL
I WANT TO ...
View Account Summary
View Recent Transactions
Transfer Funds
Search News Articles
Customize Site Language
ADMINISTRATION
Edit Users
Account History - 800000 Corporate
Balance Detail
800000 Corporate
Select Account
Amount
Ending balance as of 9/21/26 6:09 AM
-$1002880526392.45
Available balance
-$1002880526392.45
10 Most Recent Transactions
Date
Description
Amount
2026-09-21 Withdraval -$1.00
2026-09-21 Withdraval -$6000.00
2026-09-21 Withdraval -$6000.00
2026-09-21 Withdraval -$6000.00
2026-09-21 Withdraval -$0.01
2026-09-21 Withdraval -$999999999.00
Credits
Account
Date
Description
Amount
1001160140 12/29/2004 Paycheck 1200
1001160140 01/12/2005 Paycheck 1200
1001160140 01/29/2005 Paycheck 1200
Screenshots
Burp Suite Com...
Altoro Mutual - ...
Screenshot (809)...
kali-linux-2025.2...
ENG
16:39

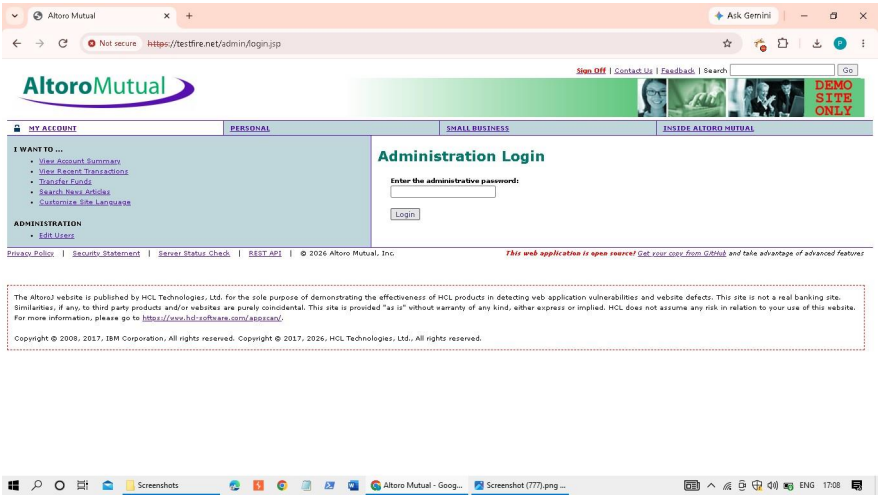
4.7 Hardcoded Administrative Password

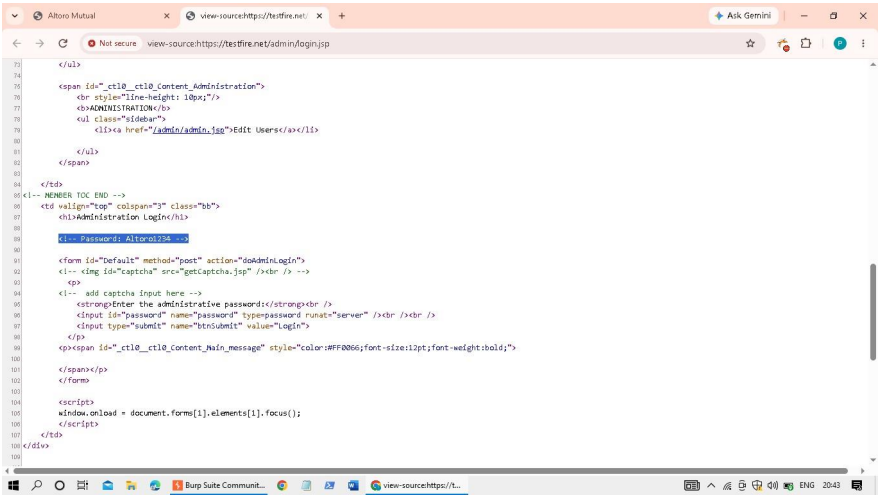
Vulnerability Name	Sensitive Information Disclosure via Hardcoded Administrative Credentials in Source Code
Vulnerability Type	Information Disclosure / Sensitive Data Exposure / Improper Credential Storage
Affected Parameter / Endpoint	Administrative login page source
Affected URL / Endpoint	https://testfire.net/admin/login.jsp
Risk Rating	**Critical**
Severity note	Summary table: High; detailed finding: Critical.
Impact	An attacker can inspect the page source, extract administrative credentials, and obtain full access to the administration console. With access to /admin/admin.jsp, an attacker can modify existing user passwords, add malicious administrative accounts, and completely compromise the application.
Remediation	Store secrets securely using environment variables or dedicated secret management systems.
CWE ID	CWE-613 / CWE-798 (Use of Hard-coded Credentials) & CWE-615 (Inclusion of Sensitive Information in Source Code Comments).

Step-by-Step Proof of Concept (PoC)

1. Navigate to the administrative login endpoint: https://testfire.net/admin/login.jsp.
2. View the page source code by pressing Ctrl + U or prepending view-source: to the URL.
3. Locate line 78 in the HTML source code, which contains an unencrypted developer comment: .
4. Return to the login interface and enter the disclosed password in the administrative password field.
5. Click 'Login'.
6. Observation: The application authenticates the request and grants access to the "Edit User Information" admin panel (/admin/admin.jsp).

Screenshot Evidence




Altoro Mutual
view-source:https://testfire.net/
Ask Gemini
Not secure
view-source:https://testfire.net/admin/login.jsp

<br style="line-height: 10px;"/>
ADMINISTRATION
<ul class="sidebar">
Edit Users

</td>
<!-- MEMBER TOC END -->
<td valign="top" colspan="3" class="bb">
<h1>Administration Login</h1>
<!-- Password: Altoro1234 -->
<form id="Default" method="post" action="doAdminLogin">
<!--
 -->

<!-- add captcha input here -->
Enter the administrative password:
<input id="password" name="password" type=password runat="server" />

<input type="submit" name="btnSubmit" value="Login">

</form>
<script>
window.onload = document.forms[1].elements[1].focus();
</script>
</td>

Burp Suite Communit...
view-source:https://t...
ENG
20:43

4.8 Sensitive Cookie Without HttpOnly Flag

Vulnerability Name	Sensitive Cookie Missing 'HttpOnly' and 'Secure' Flags
Vulnerability Type	Insecure Session Management / Sensitive Data Exposure
Affected Parameter / Endpoint	AltoroAccounts cookie
Affected URL / Endpoint	https://testfire.net/
Risk Rating	**Medium**
Impact	Attackers can hijack account context via XSS payload or sniff cleartext cookie data over unencrypted HTTP transmissions.
Remediation	Set HttpOnly, Secure, and SameSite flags on all sensitive cookies.
CWE ID	CWE-1004: Sensitive Cookie Without 'HttpOnly' Flag / CWE-614: Sensitive Cookie in HTTPS Session Without 'Secure' Attribute.

Step-by-Step Proof of Concept (PoC)

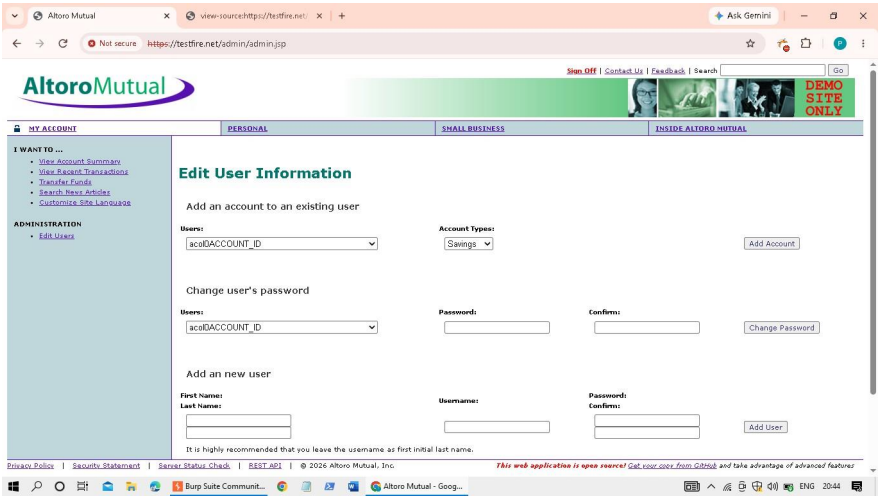
1. Authenticate to the application: https://testfire.net/login.jsp.

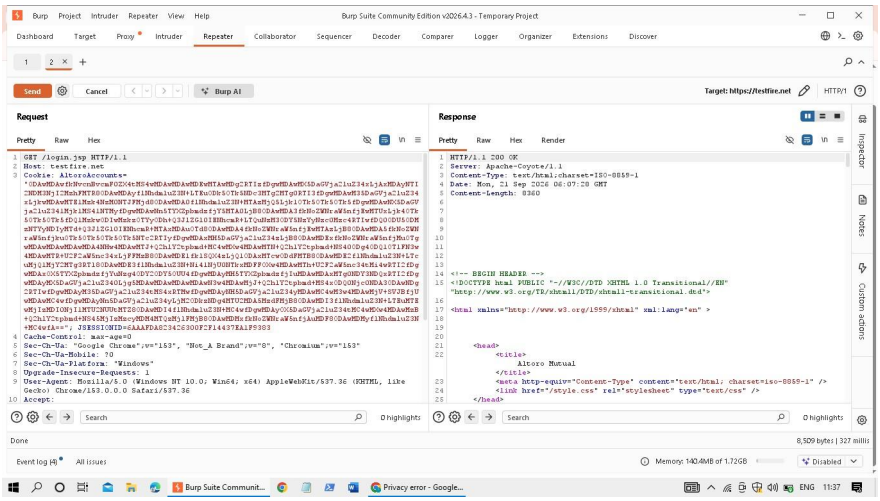
2. Open Browser Developer Tools (F12) and navigate to the Application tab.

3. Select Cookies under Storage for https://testfire.net.

4. Observation: Note that the AltoroAccounts cookie lacks both HttpOnly and Secure flag attributes, whereas JSESSIONID properly enforces both security controls.

Screenshot Evidence




Burp Project Intruder Repeater View Help
Burp Suite Community Edition v2026.4.3 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Send Cancel Burp AI
Target: https://testfire.net HTTP/1
Request
Pretty Raw Hex
GET /login.jsp HTTP/1.1
Host: testfire.net
Cookie: AltoroAccounts=
Accept:
Response
Pretty Raw Hex Render
HTTP/1.1 200 OK
Server: Apache-Coyote/1.1
Content-Type: text/html;charset=ISO-8859-1
Date: Mon, 21 Sep 2026 06:07:28 GMT
Content-Length: 8360
<!-- BEGIN HEADER -->
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" >
<head>
<title>
Altoro Mutual
</title>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<link href="/style.css" rel="stylesheet" type="text/css" />
</head>
Search 0 highlights
Done
8,509 bytes | 327 millis
Event log (4) All issues
Memory: 140.4MB of 1.72GB
Disabled
Burp Suite Communit...
Privacy error - Google...
ENG 11:37

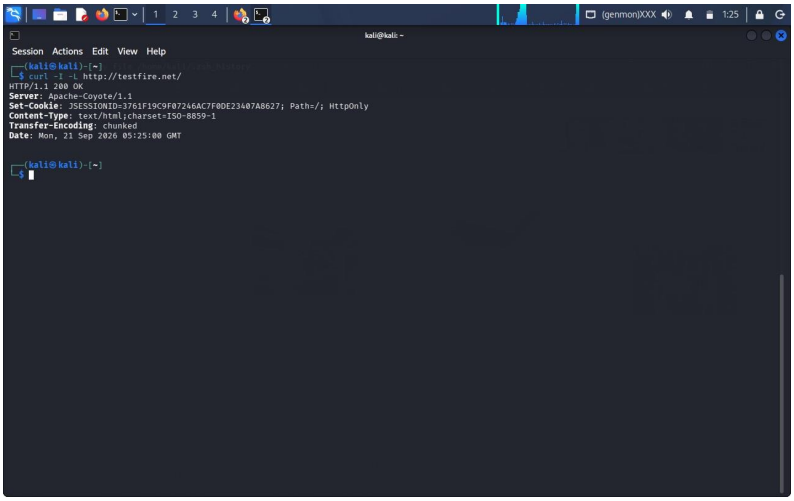
4.9 Security Header Misconfiguration

Vulnerability Name	Missing Security Response Headers
Vulnerability Type	Security Misconfiguration
Affected Parameter / Endpoint	HTTP response headers
Affected URL / Endpoint	http://testfire.net/
Risk Rating	**Low**
Severity note	Summary table: Low; detailed finding: Low or Informational.
Impact	Modern browser-side protection mechanisms are missing, making the application vulnerable to Clickjacking, MIME-sniffing, and XSS (Cross-Site Scripting) attacks.
Remediation	Configure standard security headers across all web server response responses.
CWE ID	CWE-693 (Protection Mechanism Failure) / CWE-1021 (Improper Restriction of Renderable Layers or Frames).

Step-by-Step Proof of Concept (PoC)

1. Open terminal on Kali Linux.
2. Execute the HTTP header fetch command: `curl -I -L http://testfire.net/`.
3. Observation: Review the returned HTTP response headers. Standard security headers (X-Frame-Options, Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options) are absent.

Screenshot Evidence



4.10 Missing Rate Limiting / Brute-Force Vulnerability

Vulnerability Name	Lack of Rate Limiting / Brute-Force Attack Protection
Vulnerability Type	Identification and Authentication Failures / Security Misconfiguration
Affected Parameter / Endpoint	Login authentication requests
Affected URL / Endpoint	https://testfire.net/login.jsp
Risk Rating	**Medium**
Severity note	Summary table: Medium; detailed finding: Medium / High.
Impact	Enables malicious actors to systematically guess valid user passwords or perform automated credential stuffing attacks, potentially leading to unauthorized account compromise.
Remediation	Implement Rate Limiting: Restrict the number of failed login attempts from a specific IP address, user account, or session within a given time window. Account Lockout / CAPTCHA: Temporarily lock accounts or prompt a CAPTCHA challenge after a specified threshold of consecutive failed login attempts. Exponential Backoff: Introduce progressive delays between repeated failed authentication requests.
CWE ID	CWE-307 (Improper Restriction of Excessive Authentication Attempts).

Step-by-Step Proof of Concept (PoC)

1. Navigate: Open https://testfire.net/login.jsp.

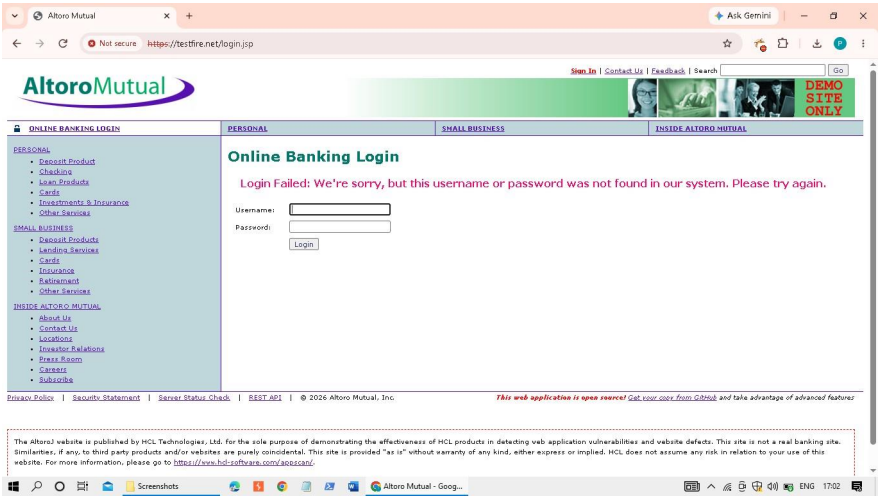
2. Intercept: Capture a login request using Burp Suite and send it to Intruder.

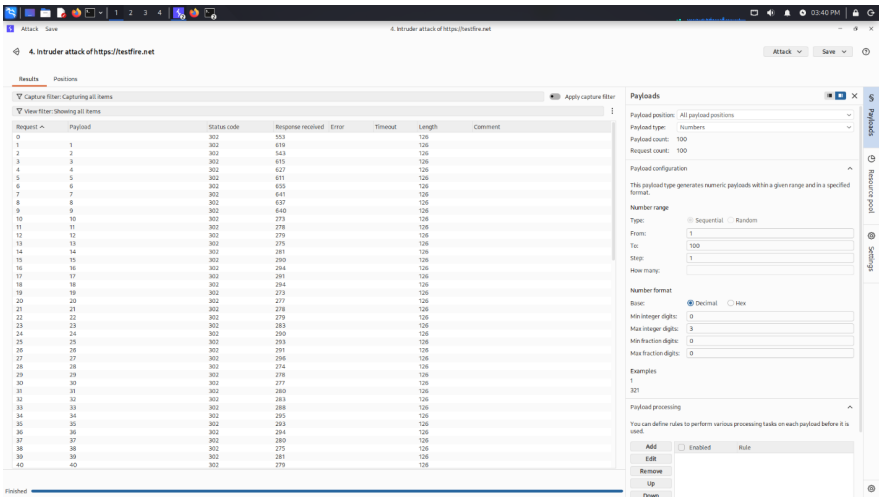
3. Configure: Set the password parameter as the payload position and select the Sniper attack type.

4. Attack: Load a password wordlist and click Start Attack.

5. Observation: The application processes all rapid password attempts without any request throttling, delay, CAPTCHA, or account lockout, confirming the absence of rate limiting.

Screenshot Evidence




4. Intruder attack of https://testfire.net
Attack
Save
Results
Positions
Payloads
Payload configuration
Payload processing
Finished

5. CONCLUSION

In conclusion, I have successfully completed the Web Application Vulnerability Assessment and Penetration Testing (VAPT) assignment provided as part of the practical training. Through hands-on testing on the target application (testfire.net), I identified, analyzed, and documented a total of 10 distinct vulnerabilities ranging from Low to Critical severity, including SQL Injection, Broken Access Control, CSRF, Reflected XSS, and missing rate-limiting controls. This assignment significantly enhanced my practical skills in identifying web security flaws using tools like Burp Suite, analyzing HTTP requests and responses, evaluating security risks, and preparing professional VAPT documentation.

Prepared By

Nikhil Gowda
Vulnerability Assessment & Penetration Testing

End of Report